

Carrera: Ingeniería en Computación

Materia: Protocolos de comunicaciones industriales

Alumnos: Bernardo Del Barco y Juan Cruz Sotelo

Docentes: Mg. Ing. Martin Pico, Ing. Milton Pozzo

IMPLEMENTACIÓN DE UNA ARQUITECTURA SCADA MULTICAPA: INTEGRACIÓN DE MODBUS RTU, OPC UA Y MQTT PARA TELEMETRÍA Y CONTROL BIDIRECCIONAL

Resumen

En este trabajo integrador se diseñó y desplegó un sistema de comunicación de datos industrial multicapa, capaz de enlazar un nodo físico de planta con una interfaz de supervisión web en la nube. El desarrollo integra el estándar Modbus RTU sobre una capa física RS485 para la adquisición de variables analógicas y digitales, delegando la construcción de la trama y el cálculo del código de redundancia cíclica (CRC) a algoritmos matemáticos a bajo nivel. Los datos adquiridos son estandarizados mediante un servidor OPC UA local y empacados en formato JSON para su transmisión global vía MQTT hacia un servidor público. Finalmente, una aplicación web asíncrona actúa como panel SCADA bidireccional, permitiendo al usuario visualizar la telemetría, comandar actuadores físicos de forma remota y monitorear el diagnóstico de fallas de la red en tiempo real. La arquitectura implementada demuestra una alta tolerancia a fallas, garantizando la reconexión automática ante cortes de conectividad física o lógica.

Palabras claves: Modbus RTU, OPC UA, MQTT, SCADA, Tolerancia a Fallos, Internet de las Cosas (IoT).

Enlace al repositorio:

https://github.com/Bernidb/Trabajo_Integrador-PCI-Del_Barco-Sotelo

1. Introducción

De acuerdo con Schwab (2016), el avance de la Industria 4.0 y el Internet de las Cosas (IoT) representa un gran desafío para la convergencia entre la tecnología de operaciones (OT) heredada y las tecnologías de la información (IT) modernas. Mientras que los protocolos seriales robustos siguen dominando el nivel de campo, la necesidad de centralizar la toma de decisiones exige la implementación de pasarelas de enlace hacia la nube para facilitar la interoperabilidad de los sistemas (Serpanos & Wolf, 2017).

En este contexto, el presente trabajo aborda la planificación, diseño y validación de una arquitectura de comunicaciones industriales estructurada en múltiples bloques funcionales. El objetivo principal es establecer un flujo de datos bidireccional desde sensores y actuadores de planta hasta la visualización en la pantalla de un usuario, integrando los protocolos Modbus, OPC UA y MQTT. Para garantizar la fiabilidad del sistema, se incorporan mecanismos de verificación de estados de comunicación y reconexión automática, respondiendo a los paradigmas y estándares de redes de información resilientes (Buyya & Dastjerdi, 2016).

2. Arquitectura del Sistema y Hardware

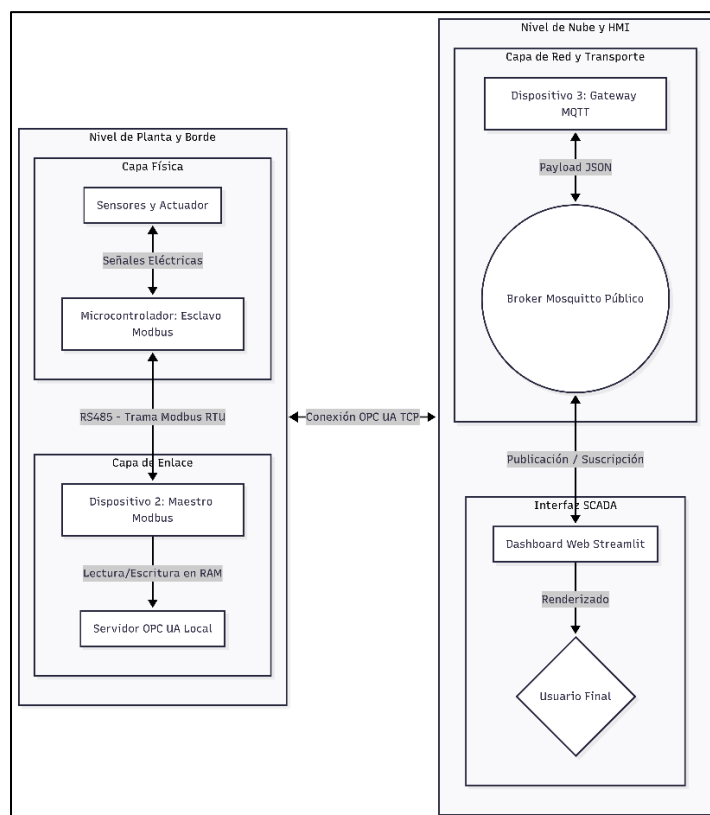
La solución propuesta se divide en cuatro bloques funcionales y lógicos que interactúan de forma secuencial y bidireccional. Esta división estructural tiene como propósito fundamental desacoplar la adquisición física de los datos de su procesamiento y visualización remota.

Por un lado, el Nivel de Planta y Borde concentra las operaciones críticas de hardware, garantizando la recolección de variables eléctricas a través del microcontrolador y su traducción a protocolos industriales estándar (Modbus RTU y OPC UA) operando de manera local y con baja latencia. Por otro lado, el Nivel de Nube y HMI actúa como la capa superior de gestión de TI (Tecnologías de la Información), donde la telemetría ya estandarizada es transmitida a través de internet mediante mensajería ligera (MQTT) para su despliegue en la plataforma SCADA interactiva.

Esta topología híbrida asegura que las tareas de lectura y control a nivel de campo no se vean bloqueadas por los tiempos de procesamiento de la interfaz de usuario, estableciendo un flujo de la información robusto y escalable. A continuación, en la Figura 1, se detalla gráficamente la arquitectura descrita.

Figura 1

Diagrama de arquitectura de red y flujo de datos del sistema.

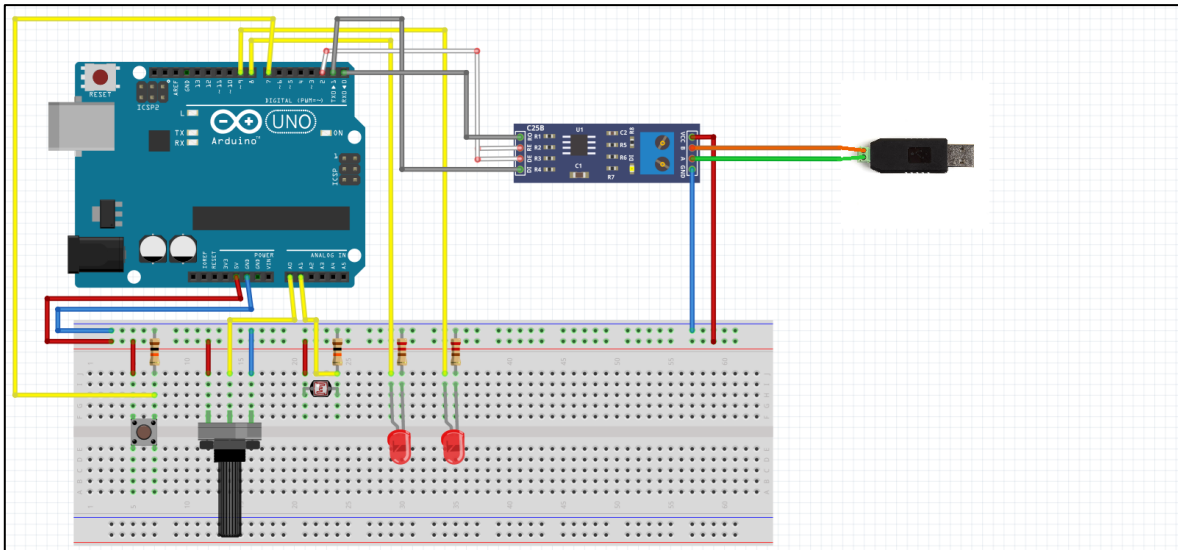


2.1. Capa Física y Dispositivo Esclavo (Dispositivo 1)

El nivel de campo está conformado por una placa de desarrollo Arduino programada como Esclavo Modbus (Dispositivo 1). La comunicación física se establece mediante el estándar diferencial RS485, requiriendo un módulo transceptor (MAX485) para convertir las señales lógicas TTL del microcontrolador. El circuito integra un potenciómetro y una fotorresistencia (LDR) para emular variaciones analógicas, junto con un pulsador digital implementado con una topología resistiva pull-down para garantizar inmunidad al ruido electromagnético en la línea y evitar corrupciones en las estadísticas de trama. Como actuador, se emplea un diodo LED que soporta tanto conmutación digital como modulación por ancho de pulsos (PWM) para variaciones analógicas. El detalle esquemático del conexionado físico de estos componentes se ilustra a continuación en la Figura 2.

Figura 2

Diagrama de conexionado físico del nodo esclavo Modbus y periféricos.



2.2. Capa de Enlace: Maestro Modbus y Servidor OPC UA (Dispositivo 2)

Este dispositivo (ejecutado en una PC) opera como el núcleo traductor del sistema. Mediante un adaptador USB-RS485, el software interroga continuamente al Dispositivo 1. Para demostrar el dominio del protocolo, la generación de la Unidad de Datos del Protocolo (PDU) y el cálculo del código CRC-16 se desarrollaron mediante la manipulación de bytes a bajo nivel, prescindiendo de librerías Modbus de alto nivel. El mismo script levanta en memoria RAM un Servidor OPC UA, mapeando los registros crudos leídos del cable físico a un árbol de variables estandarizado (Nodos), permitiendo el acceso concurrente y seguro a los datos.

2.3. Capa de Red y Transporte: Gateway MQTT (Dispositivo 3)

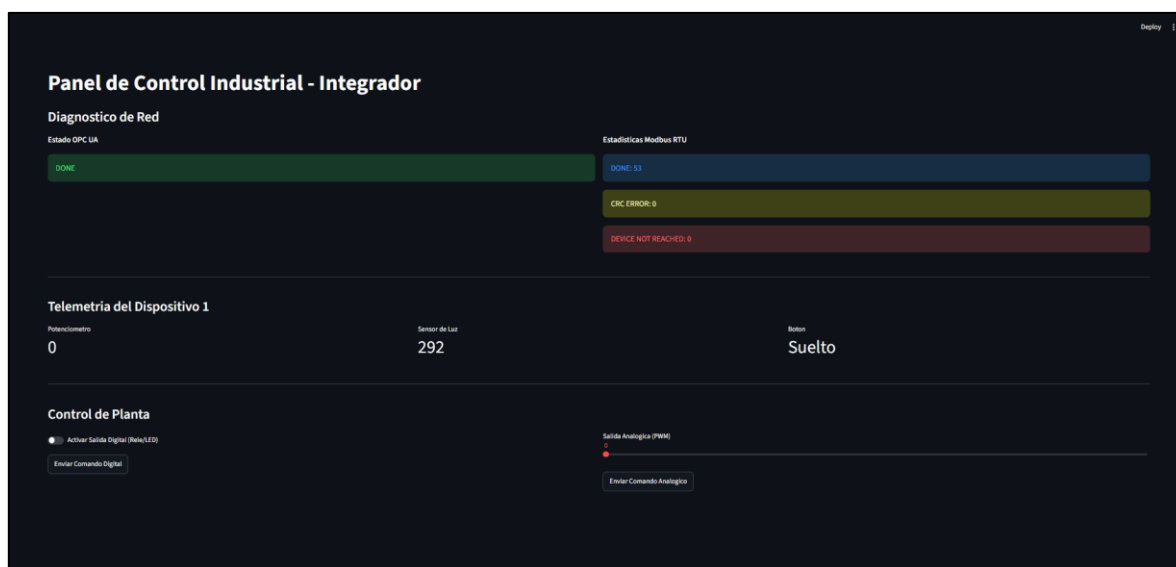
Este script opera como Cliente OPC UA y Cliente MQTT en simultáneo. Su función es suscribirse a los nodos del Servidor OPC UA local, extraer la telemetría, empaquetarla en formato estructurado JSON y publicarla a través de internet hacia un Broker MQTT público (Eclipse Mosquitto) garantizando la interoperabilidad global. A su vez, se encuentra suscrito al tópico de comandos para ejecutar el camino inverso (bajada de órdenes).

3. Interfaz SCADA y Diagnóstico

Para cumplir con los requerimientos de visualización remota y control, se desarrolló una aplicación web utilizando la plataforma Streamlit. Esta interfaz actúa como suscriptor MQTT y provee una experiencia de usuario interactiva. La comunicación bidireccional permite al usuario operar la salida digital (ON/OFF) y variar la intensidad analógica (PWM) del Dispositivo 1 desde cualquier navegador. La disposición gráfica de estos elementos de control, junto con los indicadores de telemetría en tiempo real, se expone a continuación en la Figura 3.

Figura 3

Interfaz web del sistema SCADA desarrollada con Streamlit.



Nota. La imagen evidencia el panel principal de la aplicación web, detallando la sección de comandos remotos (salidas digital y analógica), la lectura de sensores de campo y los paneles de diagnóstico de estado de red.

Para satisfacer el requerimiento de evaluación de red, el sistema recolecta y grafica constantemente el estado de las transacciones. Se implementaron tres contadores principales:

- **DONE:** Tramas Modbus enviadas y recibidas correctamente con validación CRC exitosa.

- **CRC ERROR:** Tramas recibidas con discrepancia en el cálculo de redundancia, indicando colisión o ruido en el bus RS485.
- **DEVICE NOT REACHED:** Excepciones por *Timeout* cuando el esclavo no responde en el tiempo esperado o la capa física es interrumpida.

4. Tolerancia a Fallas y Reconexión Automática

El algoritmo del Dispositivo 2 fue dotado de un sistema de control de excepciones a nivel de sistema operativo. Ante una interrupción abrupta de la capa física (ej. extracción del adaptador USB-RS485), el programa intercepta el error fatal, cierra la instancia corrupta del puerto COM de manera segura y mantiene el servidor OPC UA en línea. El bucle principal inicia rutinas de sondeo periódicas, logrando la reapertura automática del puerto y la reanudación de la comunicación en el instante exacto en que se restablece el hardware, cumpliendo satisfactoriamente con la consigna de reconexión autónoma.

5. Justificación Técnica de Librerías Empleadas

En cumplimiento con las directivas del proyecto, la siguiente tabla detalla los parámetros de configuración y funciones implementadas por librerías externas de Python.

Tabla 1. *Detalle técnico de librerías empleadas.*

Librería	Parámetros de Configuración	Funciones Empleadas e I/O
pyserial	port (String, ej. 'COM4'), baudrate (Entero, 9600), timeout (Float, 1.0 seg).	write(): Recibe bytes, retorna cantidad enviada. read(): Recibe la cantidad esperada, retorna bytes crudos.
asyncua	Endpoint (URL String, ej. "opc.tcp://0.0.0.0:4840/freeopcua/server/"), Namespace (String).	Server() / Client(): Objetos de conexión.

		add_variable(): Requiere namespace, nombre y valor inicial.
paho-mqtt	CallbackAPIVersion.VERSION1 (Enum), Broker (String, URL pública), Port (Entero, 1883).	connect(): Exige Broker, Puerto y KeepAlive. Establece sesión TCP.
struct	Configuración de formato nativa (ej. '>BBHH' para Big Endian).	pack(): Exige formato y variables. Retorna array de bytes. unpack(): Exige formato y bytes. Retorna variables desagrupadas.

6. Conclusiones

El desarrollo integral de este proyecto demostró el funcionamiento empírico del modelo OSI aplicado al entorno industrial. La creación manual de tramas Modbus validó el entendimiento exhaustivo de la capa de enlace y física, garantizando el conocimiento necesario para diagnosticar ruido eléctrico o atenuación de señales mediante el análisis de código CRC.

El encapsulamiento de estas señales de bajo nivel hacia OPC UA y su posterior publicación por MQTT permitieron aislar la criticidad del hardware local de las tecnologías de interfaz, logrando crear una arquitectura SCADA escalable, resiliente ante desconexiones físicas y totalmente operativa a través de internet.

7. Referencias

- Buyya, R., & Dastjerdi, A. V. (Eds.). (2016). *Internet of Things: Principles and Paradigms*. Morgan Kaufmann.

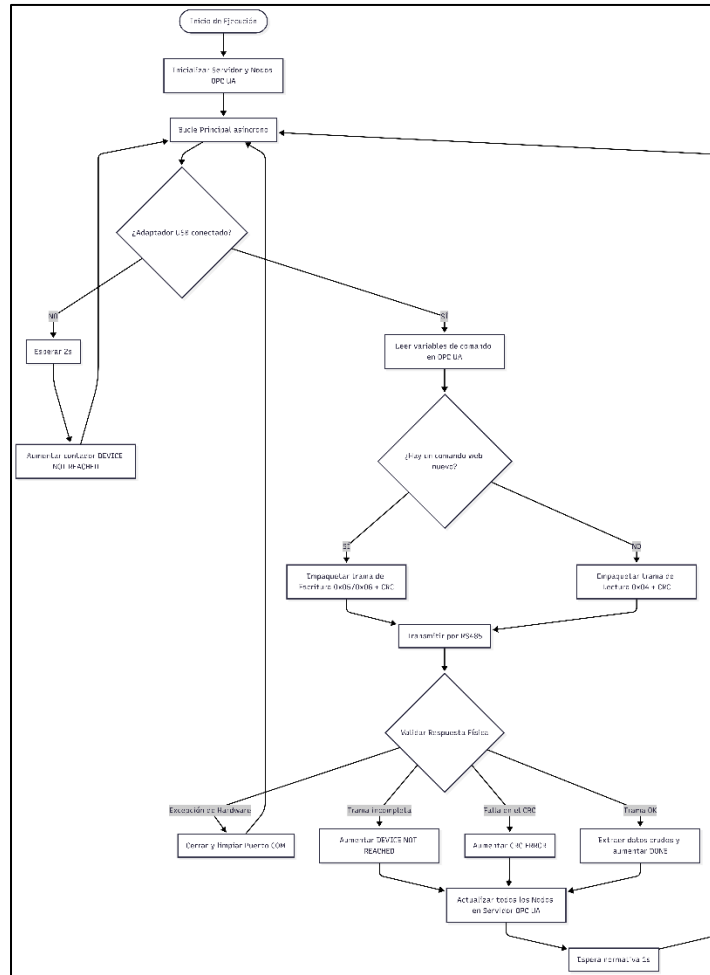
- Google. (2026). *Gemini* (Versión del 12 de agosto) [Modelo de lenguaje grande]. <https://gemini.google.com>
- Modbus Organization. (2012). *Modbus Application Protocol Specification V1.1b3*. Modbus.org.
- OASIS. (2014). *MQTT Version 3.1.1*. OASIS Standard.
- OPC Foundation. (2017). *OPC Unified Architecture Specification: Part 1 - Overview and Concepts*. OPC Foundation.
- Peterson, L. L., & Davie, B. S. (2011). *Computer Networks: A Systems Approach* (5th ed.). Elsevier.
- Schwab, K. (2016). *The Fourth Industrial Revolution*. World Economic Forum.
- Serpanos, D., & Wolf, M. (2017). *Internet-of-Things (IoT) Systems: Architectures, Algorithms, Methodologies*. Springer.
- Streamlit. (2024). *Streamlit library documentation*. <https://docs.streamlit.io/>

8. Anexos: Lógica de Ejecución y Código Fuente

Para comprender el orden de ejecución y la tolerancia a fallos del sistema, se detalla a continuación el diagrama de flujo lógico perteneciente al núcleo de la arquitectura (Dispositivo 2), seguido de los fragmentos de código fuente referenciados a sus respectivos bloques funcionales.

Figura 4

Diagrama de flujo lógico y control de excepciones del Dispositivo 2 (Maestro Modbus / Servidor OPC UA).



Nota. El diagrama ilustra el bucle principal de interrogación, destacando las rutinas de validación de hardware (adaptador USB), la verificación de integridad de datos (CRC) y la actualización de estadísticas de red.

8.1. Dispositivo 2: Validación de Capa Física

```

if puerto_serial is None or not puerto_serial.is_open:
    try:
        puerto_serial = serial.Serial(port=PUERTO_COM, baudrate=9600, timeout=1.0)
        print(f"--- PUERTO SERIAL {PUERTO_COM} CONECTADO ---")
    except serial.SerialException:
        # Si no encuentra el USB, marca error y espera antes de reintentar
        print(f"[FALLA FÍSICA] Adaptador USB no detectado en {PUERTO_COM}. Reintentando...")
        stats["DEVICE_NOT_REACHED"] += 1
        await var_stats_nr.write_value(stats["DEVICE_NOT_REACHED"])
        await asyncio.sleep(2)
        continue
  
```

Referencia al diagrama: Este fragmento corresponde al bloque de decisión "¿Adaptador USB conectado?". Si la condición es negativa, el flujo se desvía al bloque "Aumentar contador DEVICE NOT REACHED", evitando el colapso del servidor OPC UA y garantizando la reconexión autónoma exigida en los requisitos del sistema.

8.2. Dispositivo 2: Empaquetado y Cálculo de Integridad (CRC)

```
def calcular_crc(datos):  
    """  
    Implementación matemática del algoritmo CRC-16 estandar para Modbus.  
    Recorre cada byte y aplica los shifts lógicos y la compuerta XOR con 0xA001.  
    """  
    crc = 0xFFFF  
    for byte in datos:  
        crc ^= byte  
        for _ in range(8):  
            if crc & 0x0001:  
                crc = (crc >> 1) ^ 0xA001  
            else:  
                crc >>= 1  
    # Empaqueta el entero de 16 bits en 2 bytes Little Endian (Low Byte primero)  
    return struct.pack('<H', crc)
```

Referencia al diagrama: Este algoritmo matemático se ejecuta internamente dentro de los bloques "Empaquetar trama de Escritura... + CRC" y "Empaquetar trama de Lectura... + CRC". Cumple con la exigencia de generación y verificación manual del código de redundancia cíclica sin depender de librerías Modbus de alto nivel.

8.3. Dispositivo 2: Validación de Respuesta y Diagnóstico Modbus

```
estado, respuesta = enviar_y_recibir(puerto_serial, trama_peticion, 11)

if estado == "TIMEOUT":
    stats["DEVICE_NOT_REACHED"] += 1
    print("[ERROR] Timeout: Capa física desconectada.")
elif estado == "CRC_ERROR":
    stats["CRC_ERROR"] += 1
    print("[ERROR] Fallo CRC: Ruido electromagnético en la línea.")
elif estado == "OK":
    stats["DONE"] += 1

    datos_puros = respuesta[3:9]
    pot, luz, boton = struct.unpack('>HHH', datos_puros)
```

Referencia al diagrama: Este bloque condicional materializa el rombo de decisión "Validar Respuesta Física". Dependiendo de la variable estado, el código deriva el flujo hacia los bloques terminales de diagnóstico ("Aumentar CRC ERROR", "Aumentar DEVICE NOT REACHED" o "Extraer datos crudos y aumentar DONE"), actualizando las estadísticas de comunicación que luego serán visibles en la web.